DAT62-2

Neural Networks

From the mathematics of Math S4 to working PyTorch code

Every hour is spent building, training, and debugging — not re-explaining theory.

Albert School — 22 hours

Preface

You already know what a neural network **is**. In Math S4 you derived the forward pass as a chain of affine maps and nonlinearities, you turned the chain rule into backpropagation, you traced how gradients flow (and vanish), you met the attention mechanism, and you saw why regularization keeps a model honest. In Math S6 you learned the optimizers that drive learning: gradient descent, its stochastic cousin, and Adam. From DAT42-1 you can already build a complete supervised pipeline — splitting data, tuning hyperparameters, handling class imbalance, and comparing model architectures with a written report — using decision trees, random forests, and gradient boosting.

This course closes the last gap between **understanding** a network and **running** one. We never re-derive backpropagation; we let PyTorch's autograd do it and learn to trust it. We never re-explain what a gradient is; we read it off a training curve and decide what to change. The matrix product $Wx + b$ you wrote on paper becomes a `nn.Linear` layer; the loss you minimized by hand becomes a three-line training loop. Then we go beyond Math S4′s syllabus to the one architecture it does not cover — the convolutional network — and to the practical craft it prepares but never practises: diagnosing a training run that is going wrong and fixing it.

By the end you will implement a feedforward network and a CNN from scratch in PyTorch, adapt a pretrained ResNet to a new task, read pathologies off loss curves and remediate them, walk through the Transformer as an architecture, and decide — with evidence — when a neural network is the right tool and when a tree from DAT42-1 still wins. The final deliverable is a complete project with a model card you can defend to a technical audience.

A note on how to read this book: do not just read it. Every code block is meant to be typed and run. The skills this course certifies are motor skills as much as intellectual ones, and they only transfer through your own keyboard.

Contents

1 Tensors, autograd, and the training loop

1.1 From a matrix formula to a running network

In Math S4 a one-hidden-layer network on an input $x \in \mathbb{R}^d$ was written

$$\hat{y} = W_2\,\sigma(W_1 x + b_1) + b_2,$$

with σ applied componentwise. Nothing in this course changes that formula. What changes is that x, W_1, b_1, W_2, b_2 become **tensors** — typed, multi-dimensional arrays that live in PyTorch and remember how they were computed.

Start with the smallest possible concrete case before any abstraction: a single linear layer mapping three features to two outputs, evaluated on one example.

```
import torch

x  = torch.tensor([1.0, 2.0, 3.0])          # shape (3,)
W  = torch.randn(2, 3)                       # shape (2, 3)
b  = torch.randn(2)                          # shape (2,)
y  = W @ x + b                               # shape (2,)
```

The `@` operator is exactly the matrix product Wx from the formula. The shapes are the whole game: a layer that maps d_{in} features to d_{out} features holds a weight of shape $(d_{\text{out}}, d_{\text{in}})$, and the rule "inner dimensions must match" is the single most common source of errors you will hit this week.

> **Common pitfall** A neural network almost never processes one example at a time. Inputs come in **batches**: a batch of n examples is a tensor of shape (n, d), not $(d,)$. With batches the convention flips to $XW^\top + b$ so that rows stay examples: `X @ W.T + b` with `X` of shape (n, d_{in}) and `W` of shape $(d_{\text{out}}, d_{\text{in}})$ gives shape (n, d_{out}). Mixing up which dimension is the batch is the *other* most common first-week error. When a shape error appears, print `.shape` on every tensor in the line before guessing.

1.2 Autograd: backpropagation you do not have to write

The reason we left NumPy behind is `autograd`. When a tensor is created with `requires_grad=True`, PyTorch records every operation applied to it into a **computation graph**. Calling `.backward()` on a scalar walks that graph in reverse — this **is** the backpropagation you derived by hand — and deposits $\partial L/\partial \theta$ into each parameter's `.grad`.

> **Worked example — autograd reproduces a derivative you can check.** Let $L = (wx - t)^2$ with $w = 2$, $x = 3$, target $t = 1$. By hand, $\partial L/\partial w = 2(wx - t)x = 2(6 - 1)(3) = 30$.

```
w = torch.tensor(2.0, requires_grad=True)
x = torch.tensor(3.0)
t = torch.tensor(1.0)
L = (w * x - t) ** 2
L.backward()
print(w.grad)        # tensor(30.)
```

> You never wrote the derivative. The graph did. Everything in this course rests on trusting this — and on checking it once, like here, so that you do.

A network is just a deeper version of this graph. The figure below is the object autograd differentiates: a feedforward network with two hidden layers.

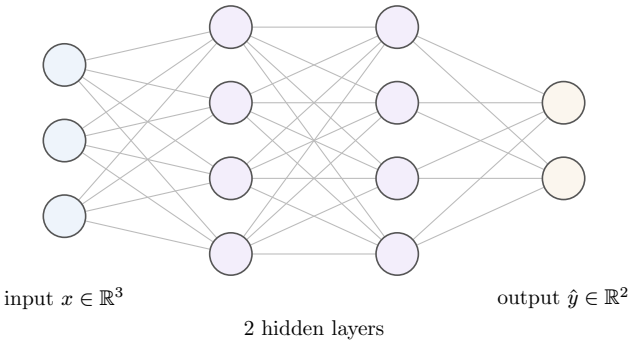

Figure 1: A feedforward network. Each edge is a weight; each node applies the nonlinearity to a weighted sum of the previous layer. Autograd differentiates this whole graph end-to-end.

1.3 Modules: the same network, written the way you will write it

Hand-multiplying matrices does not scale. PyTorch packages a layer and its parameters into an `nn.Module`. The network from the figure is:

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, d_hidden), nn.ReLU(),
            nn.Linear(d_hidden, d_hidden), nn.ReLU(),
            nn.Linear(d_hidden, d_out),
        )

    def forward(self, x):
        return self.net(x)

model = MLP(d_in=3, d_hidden=4, d_out=2)
```

`nn.Linear(d_in, d_hidden)` creates and registers the weight and bias tensors (with `requires_grad=True` already set) so the optimizer can find them through `model.parameters()`. You write only `forward`; `backward` is generated.

> **Common pitfall** The final layer outputs raw scores called **logits**. Do *not* put a `Softmax` at the end when you train a classifier with `nn.CrossEntropyLoss` — that loss applies log-softmax internally, and adding your own softmax applies it twice, silently flattening your gradients. Add a softmax only at inference time if you need probabilities.

1.4 The training loop

Math S6 gave you the update rule $\theta \leftarrow \theta - \eta \nabla_\theta L$. In PyTorch that rule is four lines, and they are always the same four lines.

```
loss_fn   = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(num_epochs):
    for xb, yb in train_loader:           # batches from a DataLoader
        optimizer.zero_grad()             # 1. clear last step's gradients
        logits = model(xb)                # 2. forward pass
        loss   = loss_fn(logits, yb)      # 3. compute the loss
        loss.backward()                   # 4. backprop -> fills .grad
        optimizer.step()                  # 5. theta <- theta - lr * grad
```

Memorize the shape of this loop; you will write it dozens of times.

> **Common pitfall** Forgetting `optimizer.zero_grad()` is the single most common silent bug in PyTorch. Gradients **accumulate** by default — `.backward()` adds to `.grad` rather than replacing it. Omit the zeroing and every step is corrupted by the sum of all previous steps' gradients; training looks like it runs but never converges. (The accumulation behaviour is occasionally useful, e.g. to simulate a large batch on a small GPU, which is why it is the default.)

Two more habits make the loop honest. Wrap evaluation in `torch.no_grad()` so you do not build a graph you will not differentiate, and toggle `model.train()` / `model.eval()` so that layers with training-specific behaviour (dropout, batch norm — Chapter 4) act correctly:

```
model.eval()
with torch.no_grad():
    for xb, yb in val_loader:
        logits = model(xb)
        # accumulate validation loss / accuracy
```

Exercises

1. Build an `MLP` for a tabular dataset you used in DAT42-1 (e.g. a binary classification problem). Train it and report accuracy. Compare to the random forest you built then — we return to this comparison rigorously in Chapter 6.

2. Deliberately remove `optimizer.zero_grad()` and plot the loss. Describe what you see and explain it in terms of gradient accumulation.

3. Take a single batch and verify by hand that `model(xb).shape` is (n, d_{out}). Change the batch size and confirm the first dimension tracks it.

2 Convolutional neural networks

2.1 Why a fully connected layer is the wrong tool for an image

Math S4 covered the feedforward network in full but stopped there. Images expose exactly why we need more. A modest 224×224 RGB image has $224 \times 224 \times 3 \approx 150\{,\}000$ numbers. A single fully connected layer mapping that to just 1000 hidden units holds $150\{,\}000 \times 1000 = 1.5 \times 10^8$ weights — for **one** layer. Worse, that layer is blind to structure: shift the cat one pixel to the right and every input changes, so a fully connected layer must re-learn "cat" separately at every position.

The convolutional layer fixes both problems with two ideas you should hold onto: **local connectivity** (a neuron looks at a small patch, not the whole image) and **weight sharing** (the same small patch detector slides across the entire image).

2.2 The convolution operation

A convolutional layer holds a small **kernel** (or **filter**), e.g. 3×3. It slides the kernel over the input; at each position it computes the dot product of the kernel with the patch underneath, producing one number. The grid of all such numbers is a **feature map**.

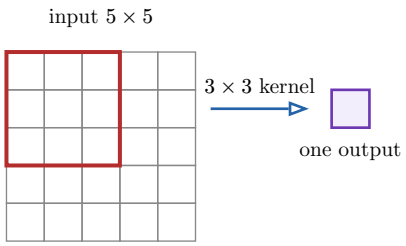

Figure 2: One step of a convolution: the 3×3 kernel covers a patch of the input (red), and its dot product with that patch becomes a single cell of the output feature map. Sliding the kernel over all positions fills the map.

With a 5×5 input and a 3×3 kernel and no padding, the kernel fits in $3 \times 3 = 9$ positions, so the output is 3×3. In general, for input size W, kernel size K, padding P and stride S:

$$W_{\text{out}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1.$$

Padding P adds a border of zeros so the output keeps the input size; **stride** S moves the kernel more than one cell at a time, shrinking the output.

A layer learns **many** kernels at once, each producing its own feature map; the stack of feature maps is the layer's output, with depth equal to the number of kernels (the number of **channels out**). In PyTorch:

```
nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
```

This learns 16 different $3 \times 3 \times 3$ kernels (the third 3 is the input depth — RGB). Note the parameter count: $16 \times 3 \times 3 \times 3 + 16 = 448$ weights, regardless of image size. Compare that to the 10^8 above.

2.3 Pooling and the shape of a CNN

Pooling downsamples a feature map. Max pooling with a 2×2 window keeps the largest value in each window, halving height and width. It buys a little translation invariance (the exact position of a feature stops mattering) and cuts computation. It has no learnable parameters.

A classic small CNN alternates convolution + nonlinearity + pooling to shrink the spatial size while growing the channel depth, then flattens and finishes with a fully connected classifier:

```
class SmallCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
```

```
            nn.Flatten(),
            nn.Linear(32 * 8 * 8, 128), nn.ReLU(),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Common pitfall The `32 * 8 * 8` in the first `Linear` is not arbitrary — it is the channels × height × width of the tensor reaching `Flatten`, and it depends on your input size and how many times you pooled. Get it wrong and you get a shape-mismatch error. The robust fix: print the shape after `self.features` once, or use `nn.AdaptiveAvgPool2d((1, 1))` to force a known size independent of the input.

2.4 Receptive fields

The **receptive field** of a neuron is the region of the original input that can influence it. A neuron in the first conv layer sees a 3×3 patch. A neuron in the second conv layer sees a 3×3 patch **of the first layer's output** — and each of those cells already summarizes a 3×3 input patch, so the second-layer neuron's receptive field is 5×5 of the original image. Pooling enlarges it faster still.

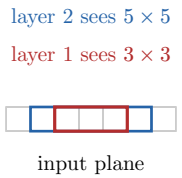

Figure 3: Stacking layers grows the receptive field. Depth, not kernel size, is how a CNN goes from detecting edges to detecting whole objects.

This is the deep insight of CNNs: early layers, with small receptive fields, learn local patterns (edges, textures); deeper layers, seeing larger regions, compose them into parts and then objects. You do not program this hierarchy — it emerges from training.

Exercises

4. For a 32×32 input passed through `Conv2d(k=3, padding=1)` → `MaxPool2d(2)` → `Conv2d(k=3, padding=1)` → `MaxPool2d(2)`, compute the spatial size at every stage using the output-size formula.

5. Train `SmallCNN` on CIFAR-10. Report test accuracy and confirm it beats a plain MLP on the same flattened images. Explain the gap using weight sharing.

6. Visualize the 16 first-layer kernels as images. Describe what kinds of local pattern they appear to detect.

3 Transfer learning

3.1 The idea: do not start from scratch

Training a strong image model from random weights needs millions of labelled images and a lot of compute — luxuries a course project does not have. **Transfer learning** reuses a network already trained on a huge dataset (ImageNet: 1.2 million images, 1000 classes) and adapts it to

your task. The early layers of such a network have already learned the generic visual vocabulary — edges, textures, shapes — that almost any vision task needs.

PyTorch's `torchvision` ships these pretrained networks ready to download:

```
from torchvision import models
backbone = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

`ResNet` introduced **residual connections** (a layer learns a correction added to its input, $x + f(x)$, rather than a full transformation) which let very deep networks train without their gradients vanishing — a direct application of the gradient-flow theory from Math S4. `EfficientNet` is a later family that scales depth, width, and input resolution together to get more accuracy per parameter. For a course project either is a fine backbone; ResNet-18 or ResNet-50 are the common defaults.

3.2 Two ways to adapt: feature extraction vs. fine-tuning

There are two regimes, and choosing between them is a learning outcome you must be able to justify.

Feature extraction. Freeze the pretrained backbone (stop its weights from updating) and replace only the final classification layer with a fresh one sized to your number of classes. You train just that small head. The backbone acts as a fixed feature extractor.

```
for p in backbone.parameters():
    p.requires_grad = False                      # freeze everything
backbone.fc = nn.Linear(backbone.fc.in_features, num_classes)  # new, trainable head
```

Fine-tuning. Start from the pretrained weights but let some or all of them keep learning on your data, usually with a **small** learning rate so you nudge the features rather than destroying them. Often you unfreeze only the later layers.

The trade-off follows two questions — how much labelled data you have, and how similar your task is to ImageNet:

	Data similar to ImageNet	**Data different**
Little data	Feature extraction — too few examples to fine-tune safely	Feature extraction from earlier layers; fine-tuning risks overfitting
Lots of data	Fine-tune the later layers	Fine-tune most of the network — you can afford to move the features

Table 1: Choosing a transfer-learning regime. With little data, freezing protects you from overfitting a giant model; with lots of data, fine-tuning lets the features specialize to your task.

Common pitfall Two preprocessing traps. (1) A pretrained model expects its inputs **normalized the same way they were during pretraining** — use the exact mean/std the weights document (`weights.transforms()` gives you the right pipeline). Feed it raw $[0, 1]$ pixels and accuracy collapses for no obvious reason. (2) When you freeze the backbone, pass **only the head's parameters** to the optimizer (`filter(lambda p: p.requires_grad, model.parameters())`), or the optimizer will happily try to update frozen tensors.

Exercises

7. Take a small image dataset (a few hundred images, a handful of classes). Train (a) `SmallCNN` from scratch and (b) a ResNet-18 via feature extraction. Compare accuracy and wall-clock training time, and explain the difference.
8. Now fine-tune the last residual block of the same ResNet with a learning rate 10× smaller than the head's. Does it beat pure feature extraction on your data? Relate your answer to the table above.

4 Diagnosing and fixing training

This is the practitioner craft that Math S4′s gradient-flow theory prepares but never lets you practise. A training run produces two curves per metric — one on the training set, one on a held-out validation set — and reading them is how you decide what to change next.

4.1 Overfitting vs. underfitting, read off the curves

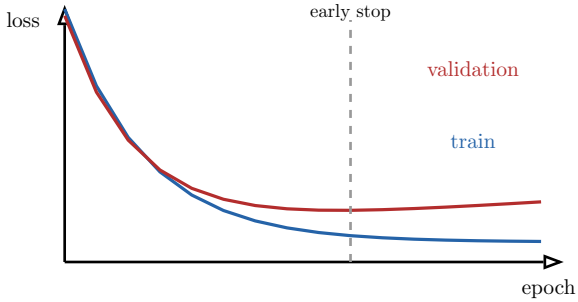

Figure 4: The signature of overfitting: training loss keeps falling while validation loss bottoms out and then rises. The gap is the model memorizing the training set. Early stopping halts at the validation minimum.

The diagnosis is a small decision table:

- **Underfitting** — both losses are high and flat. The model is too weak or has not trained long enough. Remedy: a bigger model, more epochs, a higher learning rate, or fewer regularizers.
- **Overfitting** — training loss low, validation loss high and rising (the figure). The model memorizes. Remedy: more data or augmentation, regularization (weight decay, dropout), a smaller model, or **early stopping** — keep the weights from the validation minimum.
- **Good fit** — both losses low, with a small stable gap. Stop here.

4.2 Learning rate: the one hyperparameter to get right first

The learning rate η from Math S6 governs the step size, and its symptoms are visible in the loss:

- **Too high** — loss is erratic, spikes, or diverges to `NaN`.
- **Too low** — loss decreases, but painfully slowly.
- **Just right** — a steep early drop that levels off smoothly.

A **learning-rate schedule** lowers η over time so you take big steps early and small careful steps near a minimum:

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
# ... inside the epoch loop, after optimizer.step() for the epoch:
scheduler.step()      # multiply lr by 0.1 every 10 epochs
```

4.3 Batch normalization and other stabilizers

Batch normalization normalizes each layer's pre-activations across the batch to zero mean and unit variance (then rescales with two learned parameters). It keeps activations in a healthy range, which steadies gradient flow, allows higher learning rates, and speeds up training. Insert it between a linear/conv layer and its activation: `nn.BatchNorm2d(num_channels)` for conv stacks.

> **Common pitfall** Batch norm behaves **differently** in training and evaluation: during training it uses the current batch's statistics; at evaluation it uses running averages accumulated during training. If you forget `model.eval()` before validating or deploying, batch norm (and dropout) silently use the wrong mode and your numbers are wrong. This is the practical reason the `train()`/`eval()` toggle from Chapter 1 is not optional.

Vanishing gradients — the pathology from Math S4 where gradients shrink toward zero through deep stacks, stalling the early layers — show up as early layers whose weights barely change while the loss plateaus. The modern toolkit against it is exactly what the previous chapters introduced: ReLU activations, residual connections (ResNet), batch normalization, and careful initialization.

Exercises

9. Train a model three times with learning rates 10^{-1}, 10^{-3}, 10^{-5}. Plot all three loss curves on one axis and label each with its pathology (or health).

10. Force overfitting: train a large model on a small subset with no regularization. Plot train and validation loss, identify the early-stopping epoch, then add weight decay and dropout and show the gap shrink.

11. Add `BatchNorm2d` to `SmallCNN`. Compare convergence speed (epochs to reach a target accuracy) with and without it.

5 The Transformer as a building block

Math S4 gave you the attention formula. This chapter assembles that formula into the architecture that powers modern GenAI, so that when you meet a Transformer later it is a known object, not a black box. We walk through it; we do not re-derive attention.

5.1 Recap of the one piece you already have

From Math S4, scaled dot-product attention over queries Q, keys K, values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Intuition to carry forward: each position produces a **query** asking "what am I looking for?", every position offers a **key** ("what do I contain?"), the softmax turns query–key matches into weights, and the output is the weighted blend of the **values**. **Multi-head** attention runs several such operations in parallel with different learned projections, so the model attends to several kinds of relationship at once.

5.2 Positional encoding: putting order back in

Attention is **permutation-equivariant** — shuffle the input tokens and the outputs shuffle identically, because the mechanism contains no notion of position. For language, "dog bites man" must differ from "man bites dog", so we **add** a position-dependent vector to each token

embedding before the first layer. The original Transformer uses fixed sinusoids of different frequencies:

$$\mathrm{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad \mathrm{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d}}\right),$$

where p is the position and i indexes the dimension. Different frequencies let the model read absolute and relative positions off the encoding.

5.3 The encoder block, end to end

A Transformer is a stack of identical blocks. One **encoder** block chains multi-head self-attention and a position-wise feedforward network, each wrapped in a residual connection (the $x + f(x)$ from Chapter 3) followed by layer normalization.

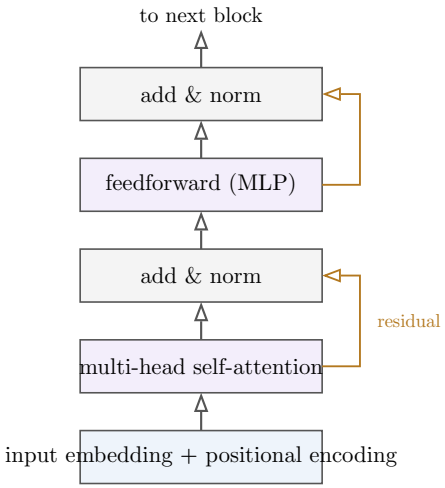

Figure 5: One Transformer encoder block. The full forward path adds positional information, mixes tokens with self-attention, then transforms each position independently with an MLP — each sublayer wrapped in a residual connection and layer norm. Stacking N such blocks gives the encoder.

The **decoder** adds two things to this template: its self-attention is **masked** so a position may attend only to earlier positions (it cannot look at the future it is trying to predict), and it inserts a **cross-attention** sublayer whose queries come from the decoder while keys and values come from the encoder's output — which is how the two stacks communicate in an encoder–decoder model. Encoder-only stacks (BERT-style) suit understanding tasks; decoder-only stacks (GPT-style) suit generation.

Remark Why the field switched to Transformers: attention relates every position to every other in a **single, fully parallel** step, whereas a recurrent network must walk the sequence one step at a time. That parallelism is what made training on internet-scale corpora feasible — the architectural reason behind the GenAI models you will use downstream.

Exercises

12. Implement scaled dot-product attention from the formula in 10 lines of PyTorch. Feed it random Q, K, V and verify the output shape and that each row of the attention matrix sums to 1.

13. Plot the sinusoidal positional encoding as a heatmap over positions and dimensions. Explain why different columns oscillate at different rates.
14. In words, trace a single token's path through the encoder block in the figure, naming the tensor shape at each stage for a sequence of length L and model dimension d.

6 When to use a neural network — and when not to

A learning outcome of both this course and DAT42-1 is choosing the right model **with justification**. Neural networks are not a universal upgrade over the trees and boosting you mastered in DAT42-1. The honest comparison rests on a few axes.

Axis	Neural network	Tree-based (RF / boosting)
Data type	Unstructured: images, text, audio — structure NNs exploit	Tabular / heterogeneous columns — still the practical default
Data volume	Shines with large datasets	Strong even on small/medium data
Interpretability	Opaque; needs post-hoc tools	Feature importances, single-tree inspection — far easier
Latency & cost	Heavier; may need a GPU	Fast to train and to serve on a CPU
Tuning effort	Architecture + many hyperparameters	Fewer, more forgiving hyperparameters

Table 2: Neural networks vs. tree-based models. On tabular data of modest size, a gradient-boosted tree from DAT42-1 is frequently the stronger **and** cheaper choice; reach for a neural network when the data is unstructured or large.

The rule of thumb worth internalizing: **unstructured data (images, text, audio) or very large datasets favour neural networks; small-to-medium tabular data favours gradient-boosted trees.** "We used deep learning" is not a justification; latency, interpretability, and data volume are. Make the argument from evidence on your actual problem — exactly the model-comparison discipline DAT42-1 asked of you, now with a neural network in the lineup.

7 The end-to-end project

The course culminates in a complete neural-network project you can defend to a technical audience. It ties together every chapter.

Workflow.

1. **Frame the problem** and pick a real dataset. State the task, the metric, and a baseline (often a tree-based model from DAT42-1 — your point of comparison).
2. **Build the data pipeline**: splits, normalization matched to any pretrained backbone, and augmentation where it helps.
3. **Choose an architecture** and justify it from Chapter 6's axes — not from fashion.
4. **Train and diagnose** with the Chapter 4 toolkit: read the curves, tune the learning rate, add regularization or early stopping as the curves demand.
5. **Evaluate** on a held-out test set with the metric you committed to up front, and compare against the baseline.
6. **Write the model card.**

What a model card contains. (1) **Intended use** — the task and who it is for. (2) **Data** — sources, size, splits, known biases. (3) **Architecture & training** — the model, key hyperparameters, the optimizer and schedule. (4) **Evaluation** — metrics on the test set, broken down across meaningful subgroups, with the baseline alongside. (5) **Limitations & ethical considerations** — where the model fails and how it could be misused. A model card is how you make an architecture choice **defensible** rather than merely **made**.

The defense. Be ready to answer: Why this architecture and not a tree-based model? What do the training curves show, and what did you change in response? Did you use transfer learning, and feature extraction or fine-tuning — why? Where does the model fail? Every one of these questions maps to a chapter of this book; if you can answer them with evidence from your own runs, you have met the course's outcomes.

Where you started, where you are. You arrived able to write the matrix form of a network and derive its gradients by hand. You leave able to **build** one in PyTorch, **train** a CNN, **adapt** a pretrained model, **diagnose** a failing run, **situate** the Transformer in the landscape of architectures, and **decide** — with evidence — whether a neural network is even the right tool. The theory was Math S4′s; the working code is yours.